

Aim

Conclusions

1. emptyDrops
2. CellBender and the droplet sweep
3. Overlap between the methods
4. What the disagreed barcodes look like
5. In the pipeline
6. What was run, in order
7. Data locations
8. Tools
9. Caveats

Benchmarking: cell calling and ambient RNA removal

Code ▾

2026-08-04

Run: 260528_VH01624_464_222K7VKNX / LK2-GEX . The comparison was carried out on LK2; CellBender was also run on LK1-resequenced.

Status: settled for the pipeline, which runs both emptyDrops and CellBender and records their overlap per run.

Analysis finished: 2026-06-29 - the most recent of the 17 figures shown below.

Report generated: 2026-08-04 by 07. cell_calling_report.Rmd , from the outputs of steps 01-06 in this folder.

Cell Ranger's own cell calling is conservative and drops small cells, T-cells in particular. Two alternatives were tested: emptyDrops, which decides whether a droplet holds more structured RNA than the ambient profile predicts, and CellBender, which models the ambient contamination and subtracts it.

For full report, download `cell_calling_report.html` in the attachment and open in a browser.

Code used - attached and in the github repository:

https://github.com/nbartonicek/surface_proteome_dev/tree/main/lab_archives/Benchmarking-cell_calling_and_ambient_RNA_removal

(https://github.com/nbartonicek/surface_proteome_dev/tree/main/lab_archives/Benchmarking-cell_calling_and_ambient_RNA_removal)

Aim

Decide which barcodes to carry forward as real cells, and how much ambient RNA to remove:

- does emptyDrops recover the small cells Cell Ranger discards

- does CellBender agree with emptyDrops on which barcodes are cells
- what `expected-cells` and `total-droplets-included` should CellBender use
- what do the barcodes the two methods disagree about actually look like

Conclusions

- Cell Ranger is fairly conservative and eliminates small cells, T-cells in particular; emptyDrops recovers roughly 1,000 more cells on run 1 (24,334)
- run 2 raises the opposite problem: emptyDrops calls **fewer** cells than Cell Ranger (31,198 against 32,846), and the extra barcodes are largely dead cells - so more calls is not automatically better
- the working combination is emptyDrops plus CellBender with a mitochondrial cleanup, rather than either tool alone
- MT cutoffs are taken from the literature rather than picked here: van Galen 20%, Petti 10%, Beneyto 20%, Zeng 20%
- the fraction of counts removed stays under 5% in every configuration tried (2.3% to 4.3%), so CellBender is not stripping a large part of the signal
- the droplet padding matters less than expected: +15,000 and +30,000 total droplets give substantially overlapping barcode sets, which is what the Venn diagrams below show
- raising `total-droplets-included` raises `found_cells` (35,358 to 38,215 on LK2) while *lowering* the fraction removed, so the two settings trade off against each other rather than one dominating

1. emptyDrops

Step 01 runs `DropletUtils::emptyDrops` on the raw Cell Ranger matrix and keeps barcodes at $FDR \leq 0.01$. It estimates the ambient RNA profile from the low-count droplets and then tests, per droplet, whether its content is more structured than that profile would produce. That is what lets it recover small cells a simple UMI-rank cutoff discards.

2. CellBender and the droplet sweep

CellBender 0.3.0 was run in a container (step 02). `expected-cells` is estimated from the data; `total-droplets-included` sets how far down the UMI rank the model looks for ambient signal. Two settings were tried:

Step	<code>total-droplets-included</code>	Output
02	<code>expected + 15,000</code>	<code>cellbender/<run>/<sample>/</code>
03	<code>expected + 30,000</code>	<code>cellbender/<run>/<sample>_e30000/</code>

Those are the 32k and 32k30k arms in the figure names below.

Four configurations were run in total, three of them on LK2. The numbers below are from the CellBender metrics of each run, as reported in the 2026-07-13 progress slides:

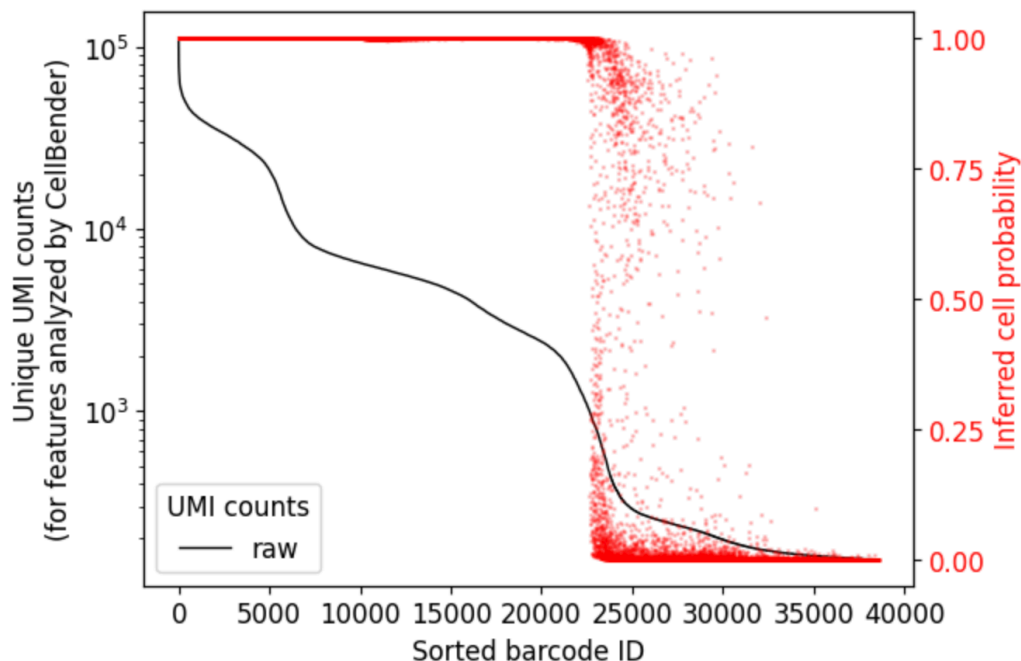
	LK1 23k+15k	LK2 33k+15k	LK2 25k+15k	LK2 33k+30k
<code>expected_cells</code>	23,569	32,846	25,000	32,846
<code>found_cells</code>	23,595	35,358	34,658	38,215

	LK1 23k+15k	LK2 33k+15k	LK2 25k+15k	LK2 33k+30k
<code>fraction_counts_removed</code>	0.023	0.034	0.043	0.031

The diagnostic below is the panel to read: the black line is the UMI count per barcode in rank order, the red points are CellBender's inferred cell probability. Where the red cloud breaks up is where the model stops being confident a droplet holds a cell.

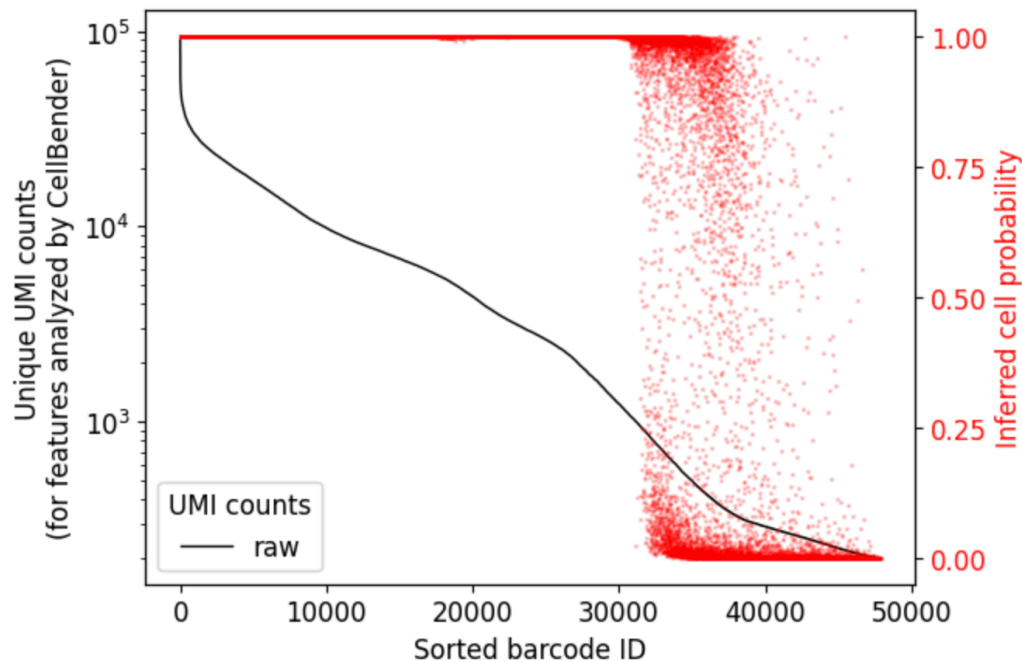
cellbender_LK1_23k_15k.png

LK1, expected 23,569 + 15,000 droplets. 23,595 cells found, 2.3% of counts removed - the cleanest of the four, and the boundary is sharp.



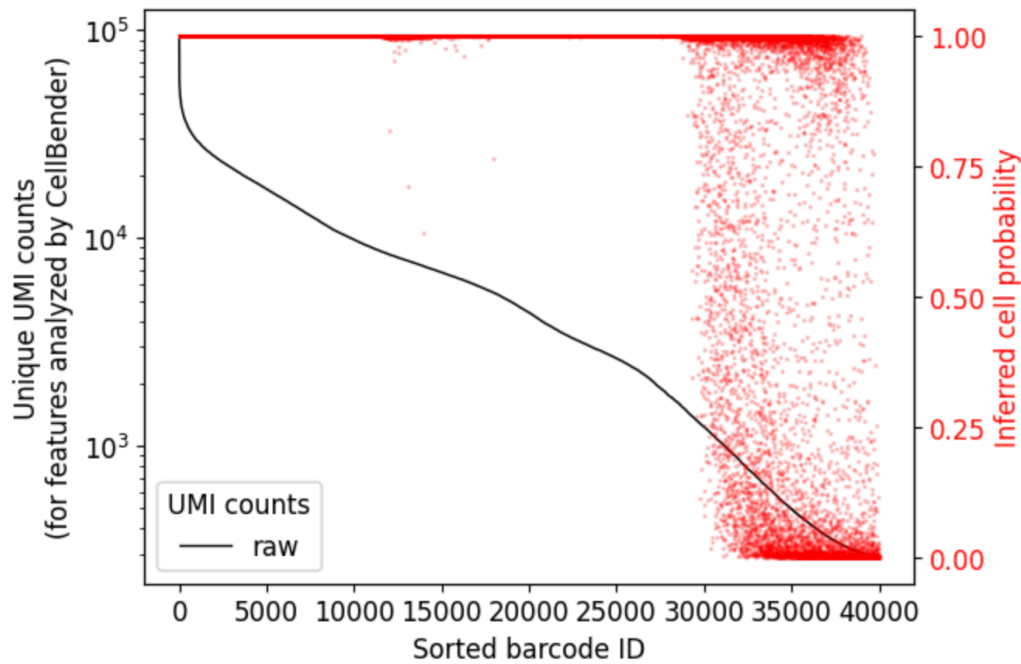
cellbender_LK2_33k_15k.png

LK2, expected 32,846 + 15,000. 35,358 cells, 3.4% removed. The adopted setting (step 02).



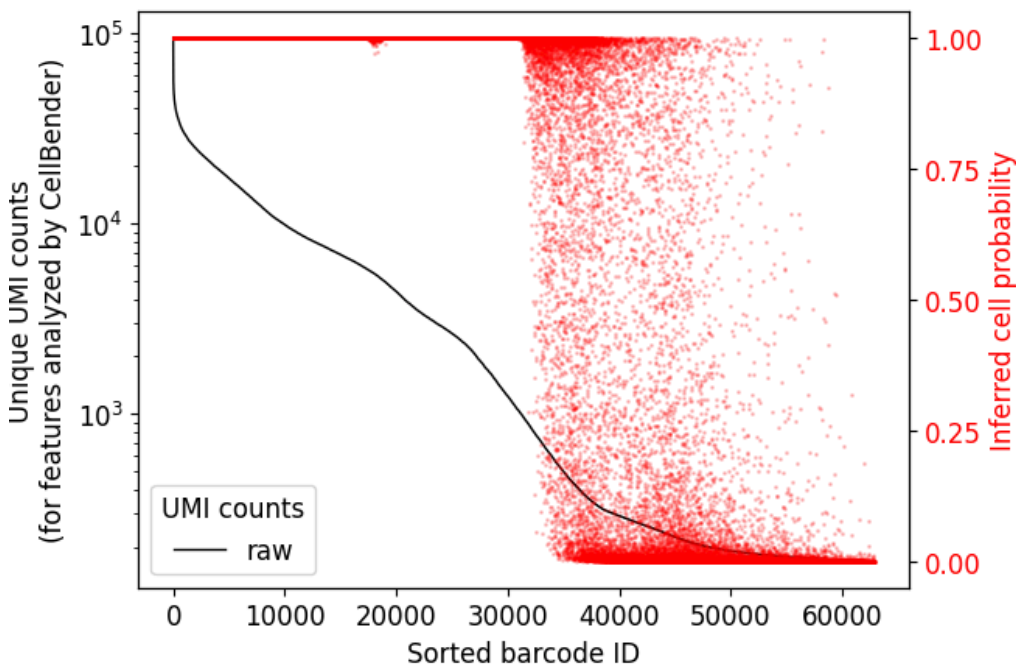
cellbender_LK2_25k_15k.png

LK2, expected 25,000 + 15,000. 34,658 cells, 4.3% removed - understating expected-cells costs the most signal of the three.

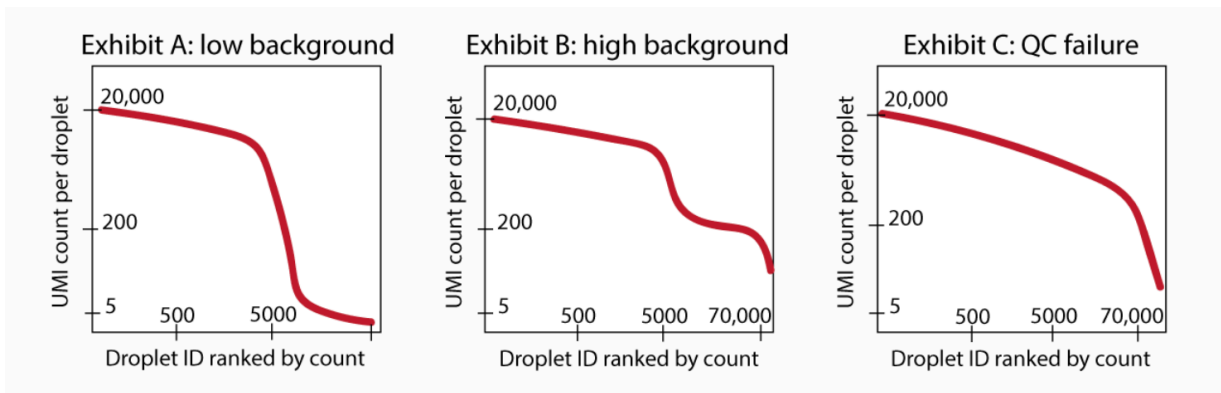


cellbender_LK2_33k_30k.png

LK2, expected 32,846 + 30,000 (step 03). 38,215 cells, 3.1% removed. Widening the droplet window finds more cells and removes proportionally less.



For reference, CellBender's own illustration of what the UMI-rank curve looks like at low background, high background, and outright QC failure:



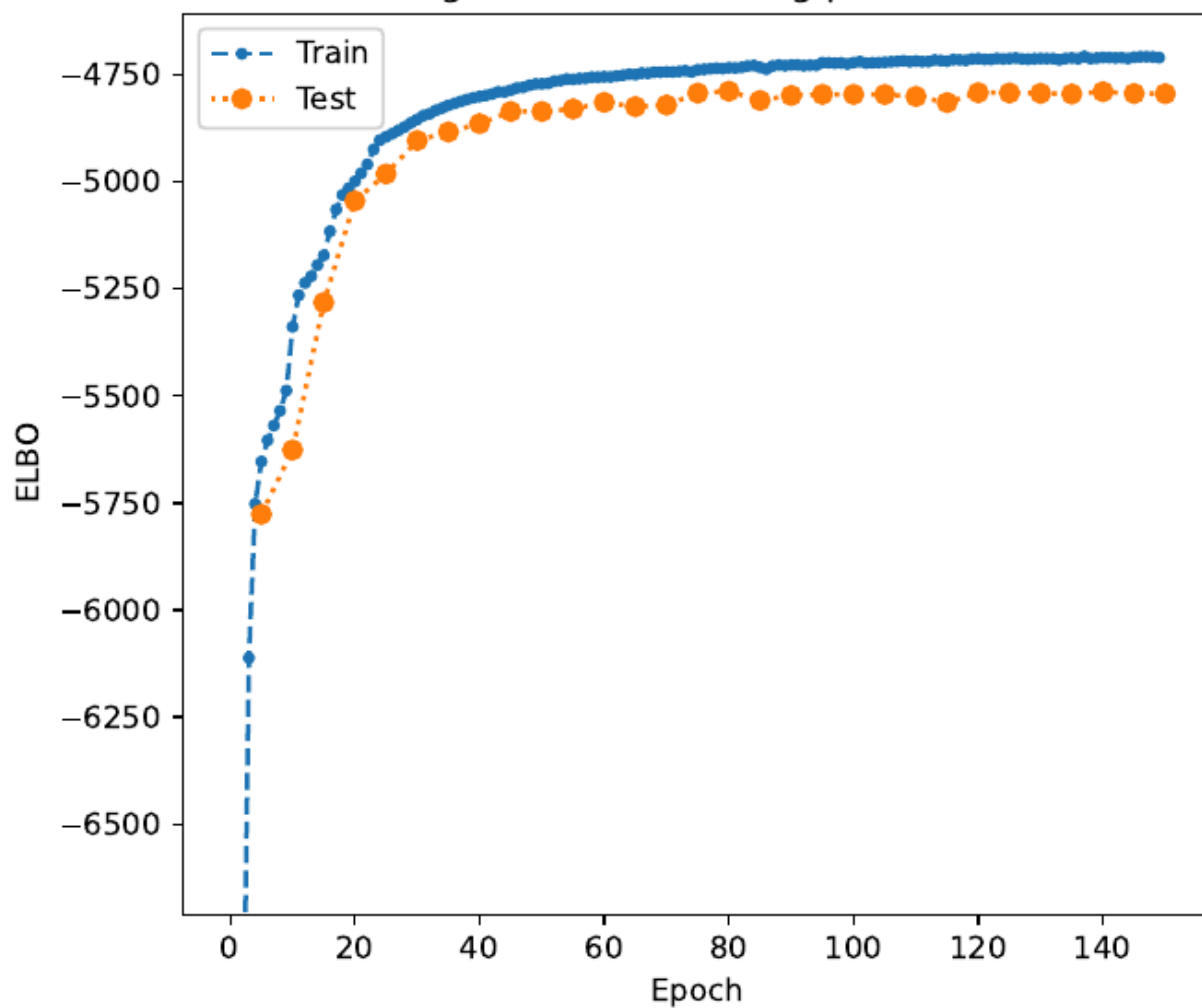
CellBender metrics

metric	value
total_output_counts	280,092,053
total_counts_removed	9,879,479
fraction_counts_removed	0.034
total_raw_counts_in_cells	289,971,532
total_counts_removed_from_cells	9,879,479
fraction_counts_removed_from_cells	0.034
average_counts_removed_per_cell	279.41
target_fpr	0.010
expected_cells	32,846
found_cells	35,358
output_average_counts_per_cell	7,922
ratio_of_found_cells_to_expected_cells	1.076
found_empties	12,488
fraction_of_analyzed_droplets_that_are_nonempty	0.739
convergence_indicator	0.177
overall_change_in_train_elbo	6,949

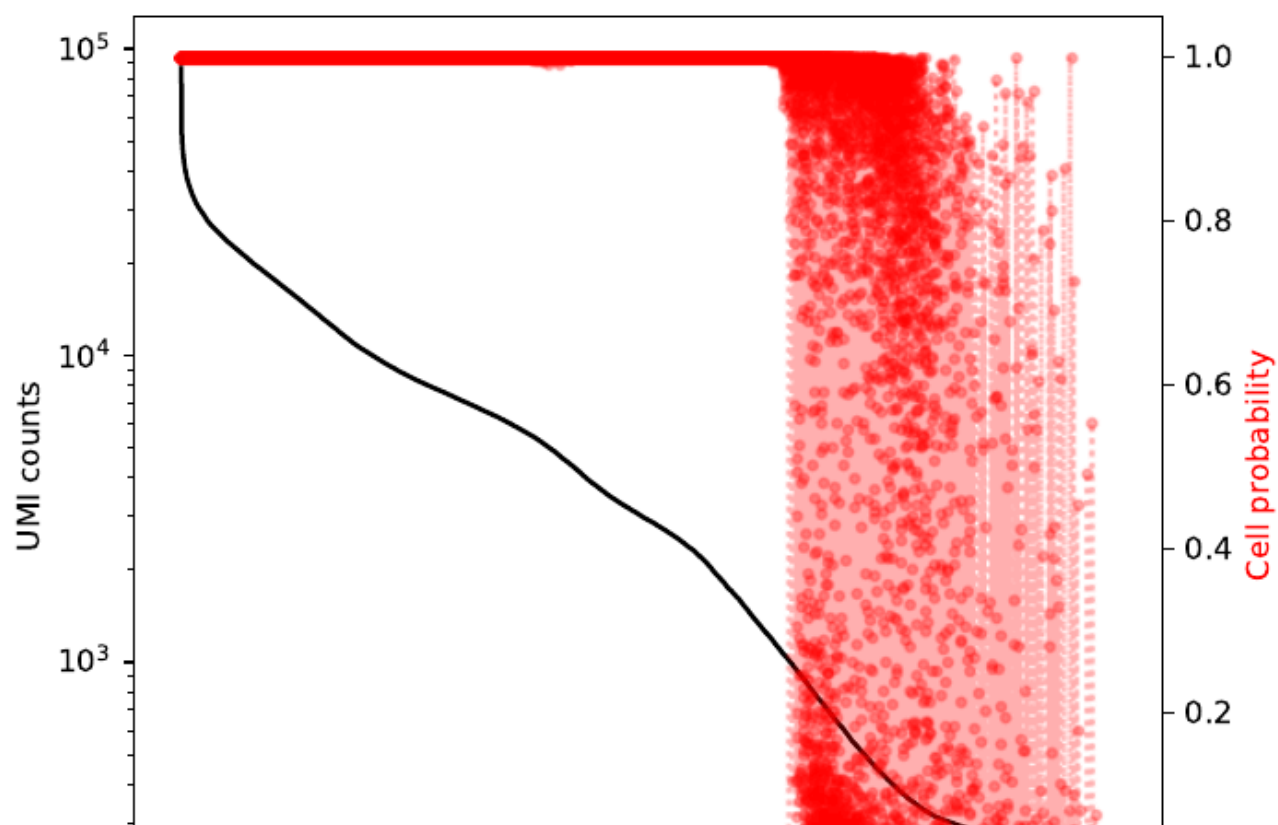
LK2-GEX_cellbender.pdf

CellBender's own diagnostic page: training progress, cell probability against UMI rank, and a PCA of the latent encoding. The middle panel is the one to read - it shows where the model puts the boundary between cells and ambient droplets.

Progress of the training procedure



Determination of which barcodes contain cells



3. Overlap between the methods

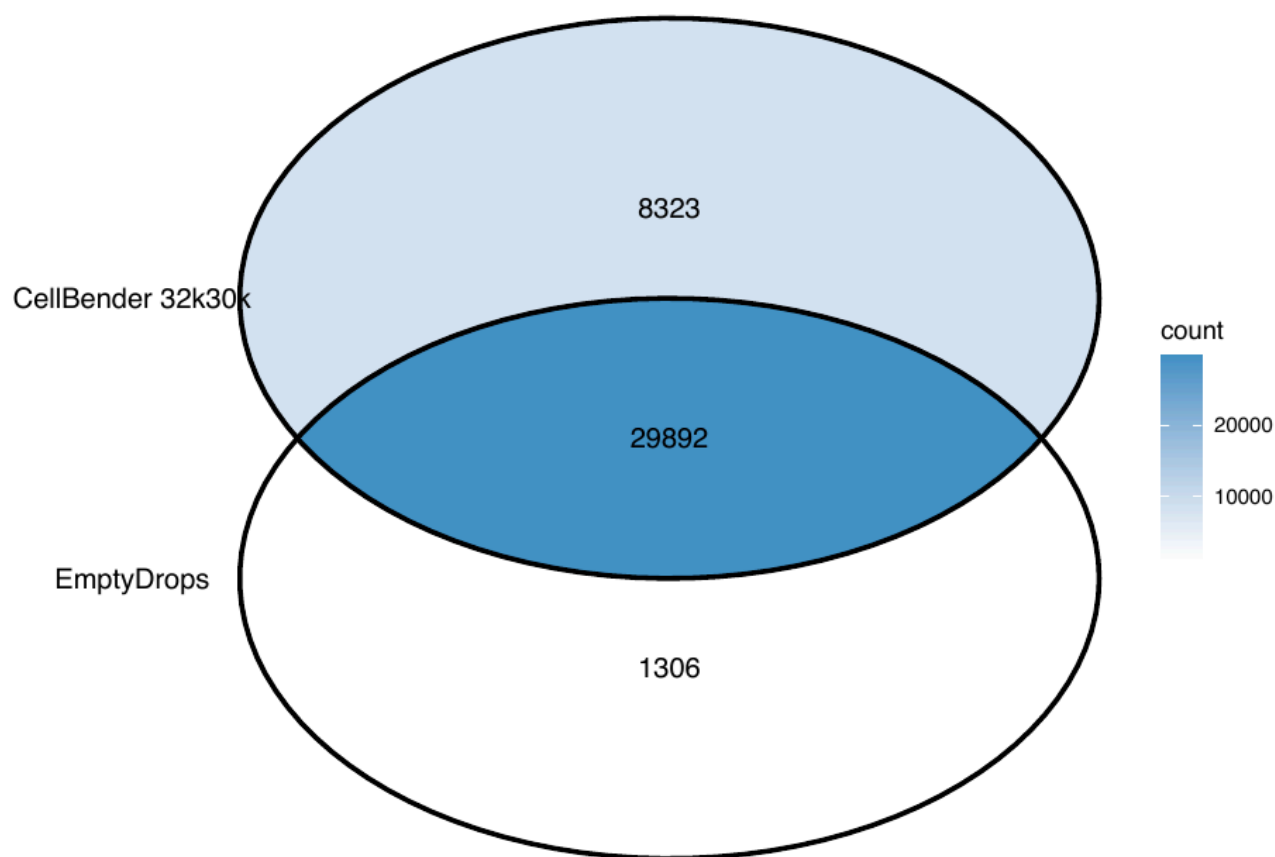
venn_emptydrops_vs_cellbender_25k.pdf
emptyDrops against CellBender at the 25k setting.
(could not rasterise venn_emptydrops_vs_cellbender_25k.pdf : PDF parsing failure.)

PCA of latent encoding of gene expression in cells

venn_emptydrops_vs_cellbender_32k.pdf
emptyDrops against CellBender at 32k (expected + 15,000 droplets).
(could not rasterise venn_emptydrops_vs_cellbender_32k.pdf : PDF parsing failure.)

venn_emptydrops_vs_cellbender_30k.pdf
emptyDrops against CellBender at 30,000 extra droplets.

Cell calling comparison – LK2–GEX



venn_cellbender_32k_vs_25k.pdf

The two CellBender settings against each other - how much the expected-cells choice actually moves the barcode set.

(could not rasterise venn_cellbender_32k_vs_25k.pdf : PDF parsing failure.)

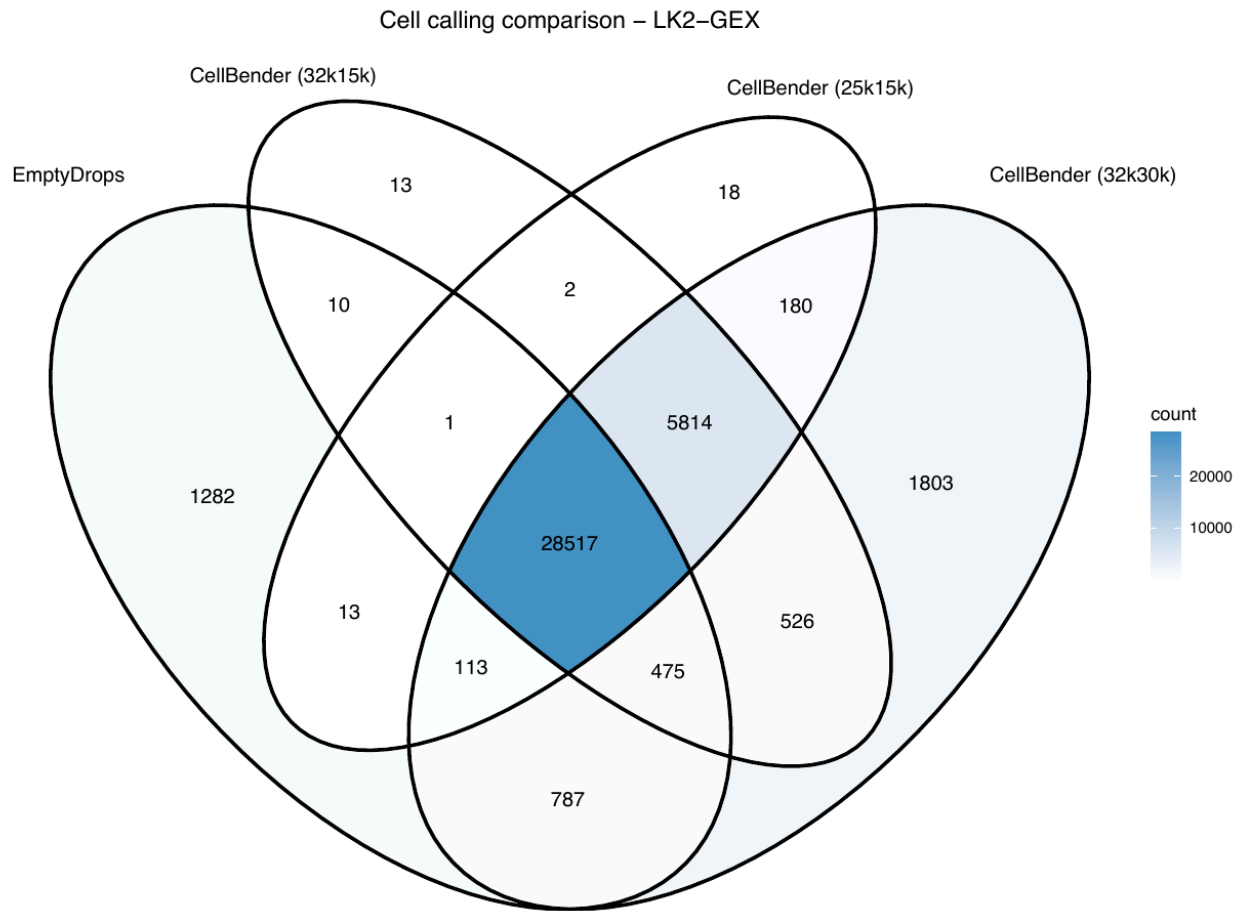
venn_emptydrops_cellbender_3way.pdf

Three-way overlap.

(could not rasterise venn_emptydrops_cellbender_3way.pdf : PDF parsing failure.)

venn_emptydrops_cellbender_4way.pdf

Four-way overlap, adding the +30k arm.



barplot_category_breakdown.pdf

How many barcodes fall in each agreement category.

(could not rasterise barplot_category_breakdown.pdf : PDF parsing failure.)

4. What the disagreed barcodes look like

The overlap alone does not say which method is right. Step 06 takes every barcode category - called by both, by only one, by neither - and looks at its QC.

QC summary by barcode category

caller	barcode_category	n_cells	median_genes	median_umis	median_mt_counts	median_percent_mt
CellBender 32k30k object	Common	29892	2635.0	6186	192	2.735794

caller	barcode_category	n_cells	median_genes	median_umis	median_mt_counts	median_percent_mt
CellBender 32k30k object	CellBender only	8323	23.0	405	150	91.760300
EmptyDrops object	Common	29892	2668.0	6375	212	2.933419
EmptyDrops object	EmptyDrops only	1306	207.5	243	6	2.750717

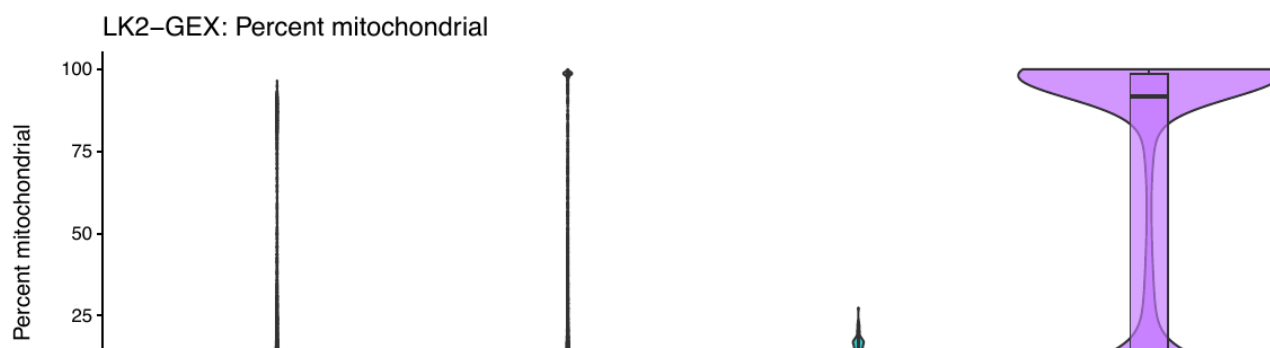
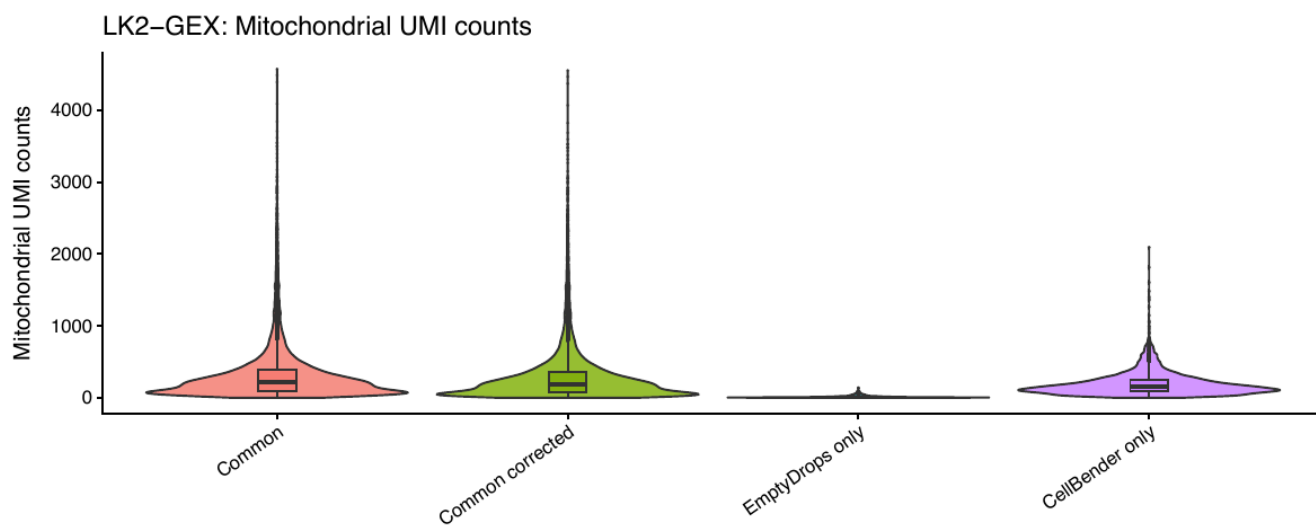
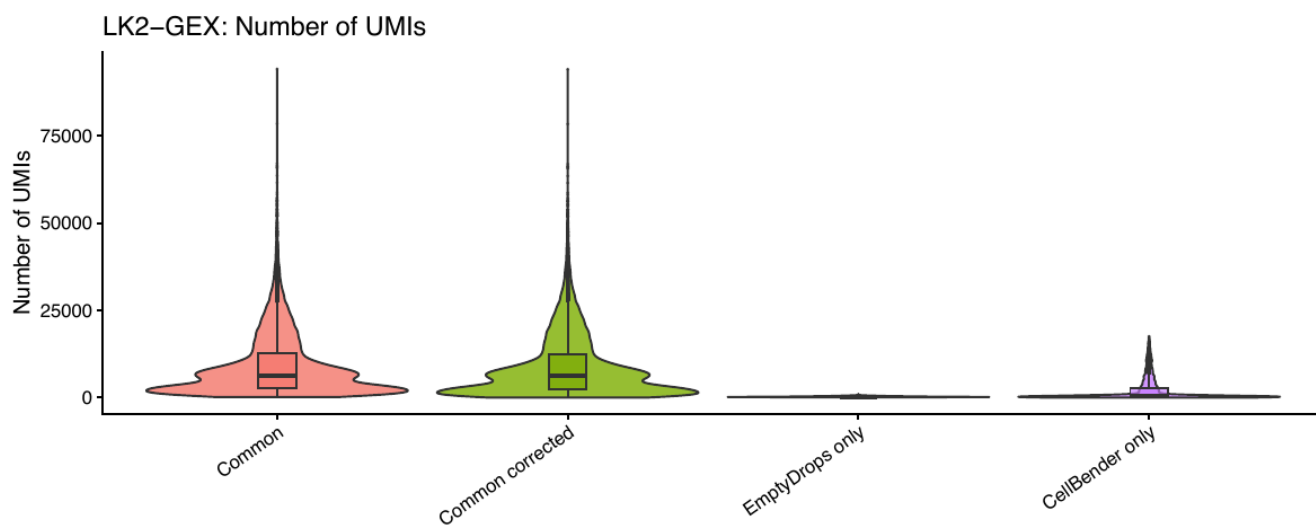
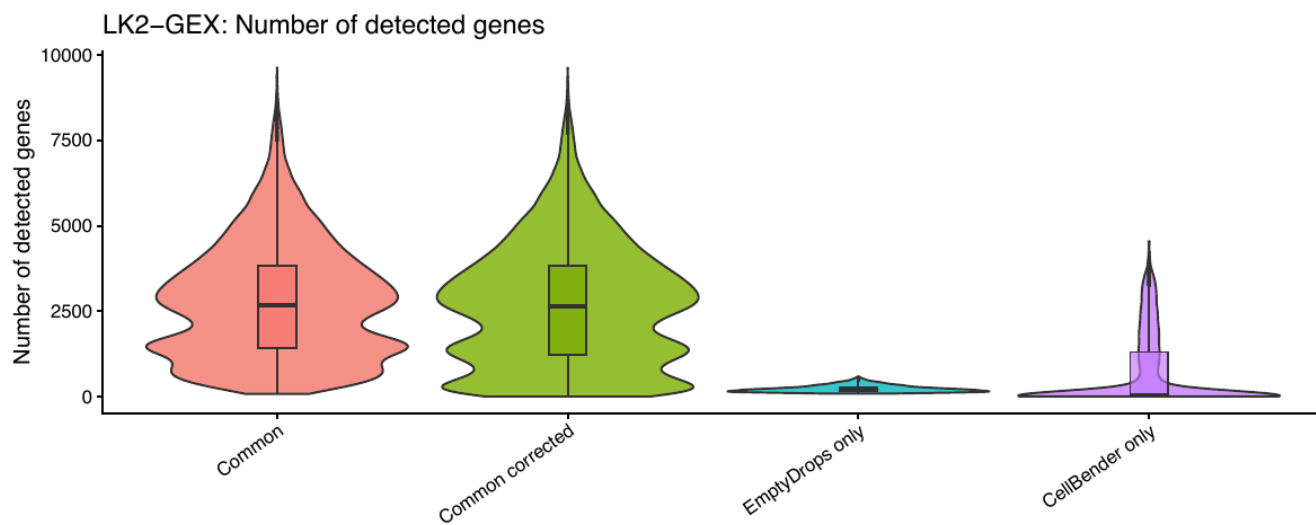
QC by barcode category

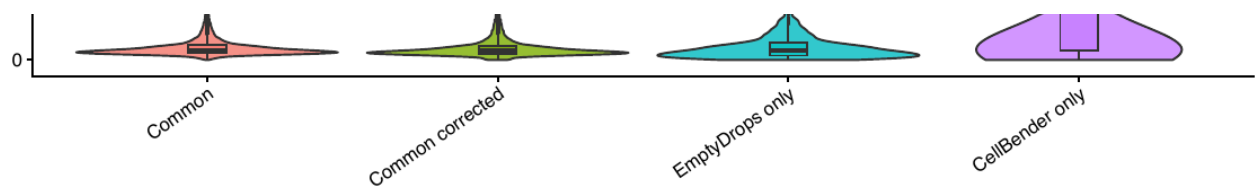
barcode	barcode_category	nFeature_RNA	nCount_RNA	nCount_MT	percent_mt
AAACCAAAGAAGGAGA-1	Common	3677	11588	313	2.7010701
AAACCAAAGCGCCTAG-1	Common	6680	25131	1569	6.2432852
AAACCAAAGGAATTAG-1	Common	5404	29863	646	2.1632120
AAACCAAAGGGCTCCA-1	Common	485	707	2	0.2828854
AAACCAAAGGTAGCTC-1	Common	4652	21572	704	3.2634897
AAACCAAAGGTTACAT-1	Common	5320	21908	412	1.8805916
AAACCAAAGTCGCGCA-1	Common	3505	12810	142	1.1085090
AAACCATTACAGGGA-1	Common	3240	7865	352	4.4755245
AAACCATTCCGGTGAT-1	Common	1916	3565	57	1.5988780
AAACCATTCTAAGTAC-1	Common	1825	3272	50	1.5281174
AAACCCGCAAACCGCA-1	Common	1472	2554	209	8.1832420
AAACCCGCAAAGGCCC-1	Common	2350	5517	124	2.2475983
AAACCCGCAACTGCGA-1	Common	2692	6682	464	6.9440287
AAACCCGCAATCCAGG-1	Common	1678	2920	92	3.1506849

barcode	barcode_category	nFeature_RNA	nCount_RNA	nCount_MT	percent.mt
AAACCCGCACTAGCAG-1	Common	657	1503	139	9.2481703
AAACCCGCACTCGCTG-1	Common	1180	1900	103	5.4210526
AAACCCGCAGTTAGGT-1	Common	1028	1746	266	15.2348225
AAACCCGCAGTTAGTA-1	Common	3243	7083	129	1.8212622
AAACCCGCATAAGCCG-1	Common	3263	8499	358	4.2122603
AAACCCGCATTACGAA-1	Common	4573	13126	491	3.7406674
AAACCCTGTAACGGTA-1	Common	3412	8507	292	3.4324674
AAACCCTGTAATGTCC-1	Common	5861	19261	1020	5.2956752
AAACCCTGTCTAGCAT-1	Common	758	1414	444	31.4002829
AAACCCTGTGAGCCCT-1	Common	5793	30181	810	2.6838077
AAACCCTGTGCAGGTG-1	Common	1227	3202	1020	31.8550906
AAACCCTGTTAGCGAA-1	Common	3099	7473	525	7.0252910
AAACCCTGTTAGCTGT-1	Common	3425	10160	219	2.1555118
AAACCCTGTTATGACC-1	Common	4184	13101	355	2.7097168
AAACCCTGTTCCCTGT-1	Common	4870	20603	443	2.1501723
AAACCCTGTTCGGGCT-1	Common	2594	6509	303	4.6550929

violin_qc_metrics_by_barcode_category_combined.pdf

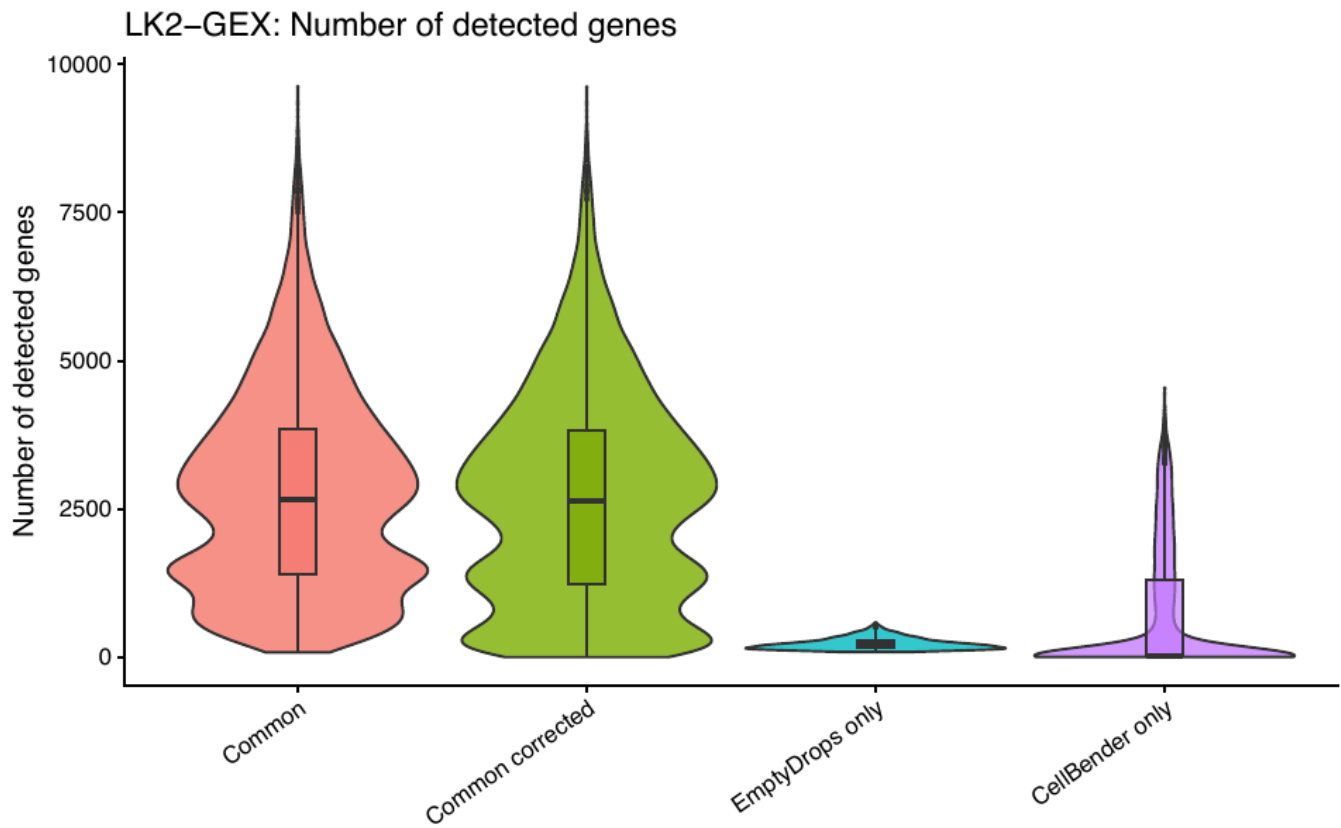
All QC metrics by barcode category in one panel. The summary figure for this section.





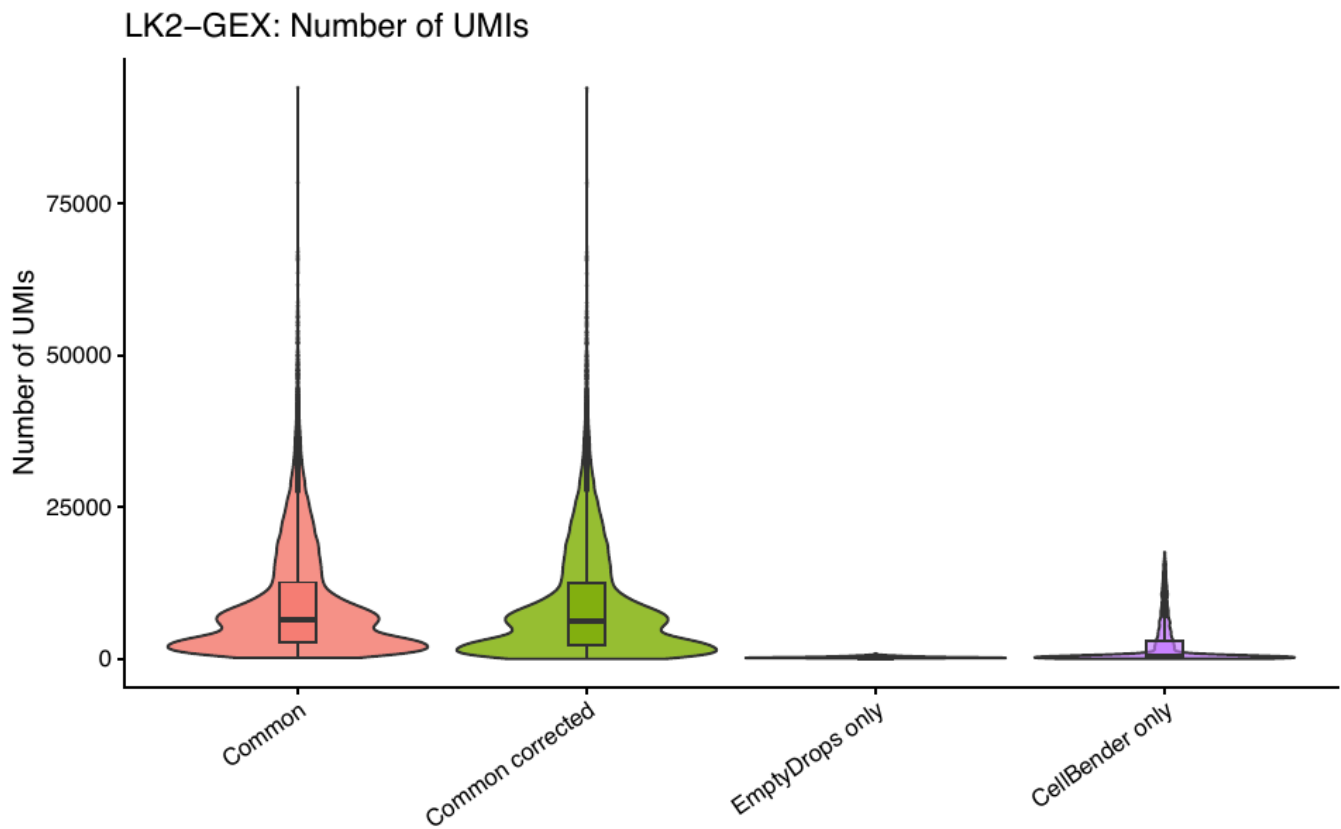
violin_genes_by_barcode_category.pdf

Genes detected per barcode category.



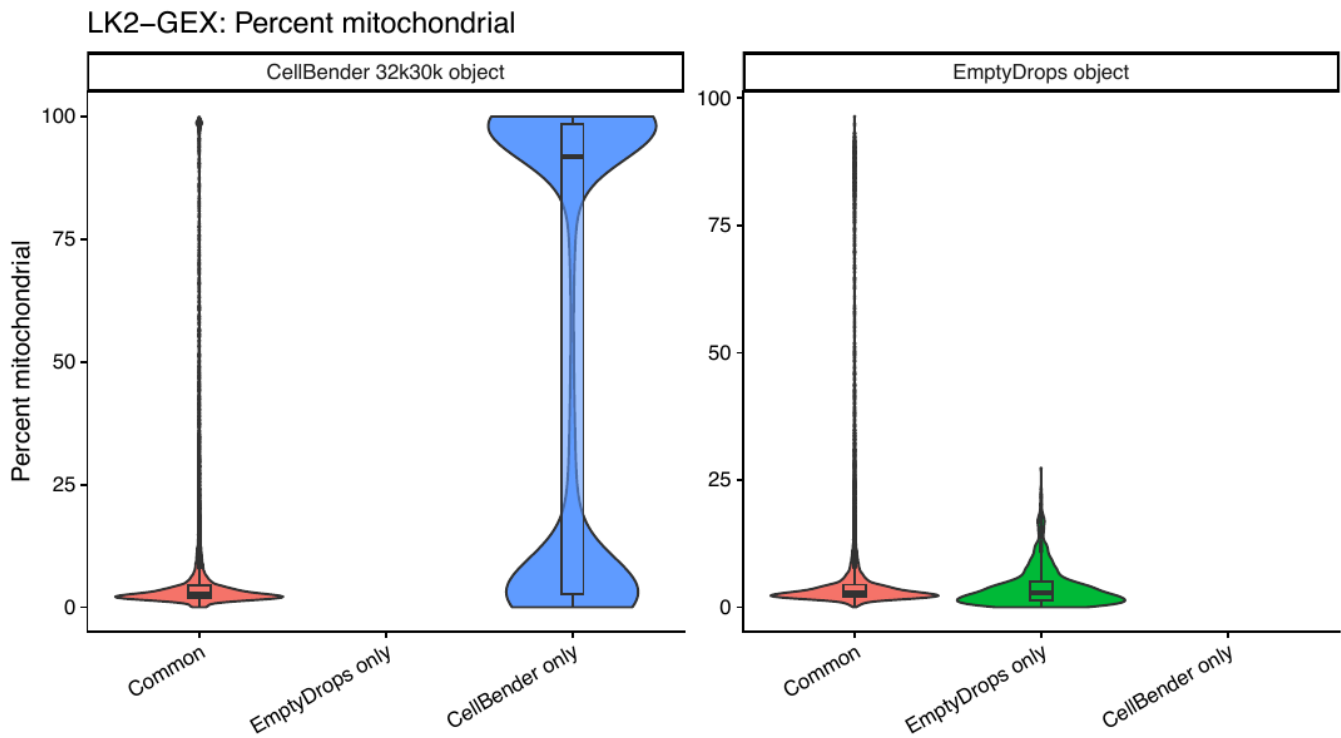
violin_umis_by_barcode_category.pdf

UMIs per barcode category.



violin_percent_mt_by_barcode_category.pdf

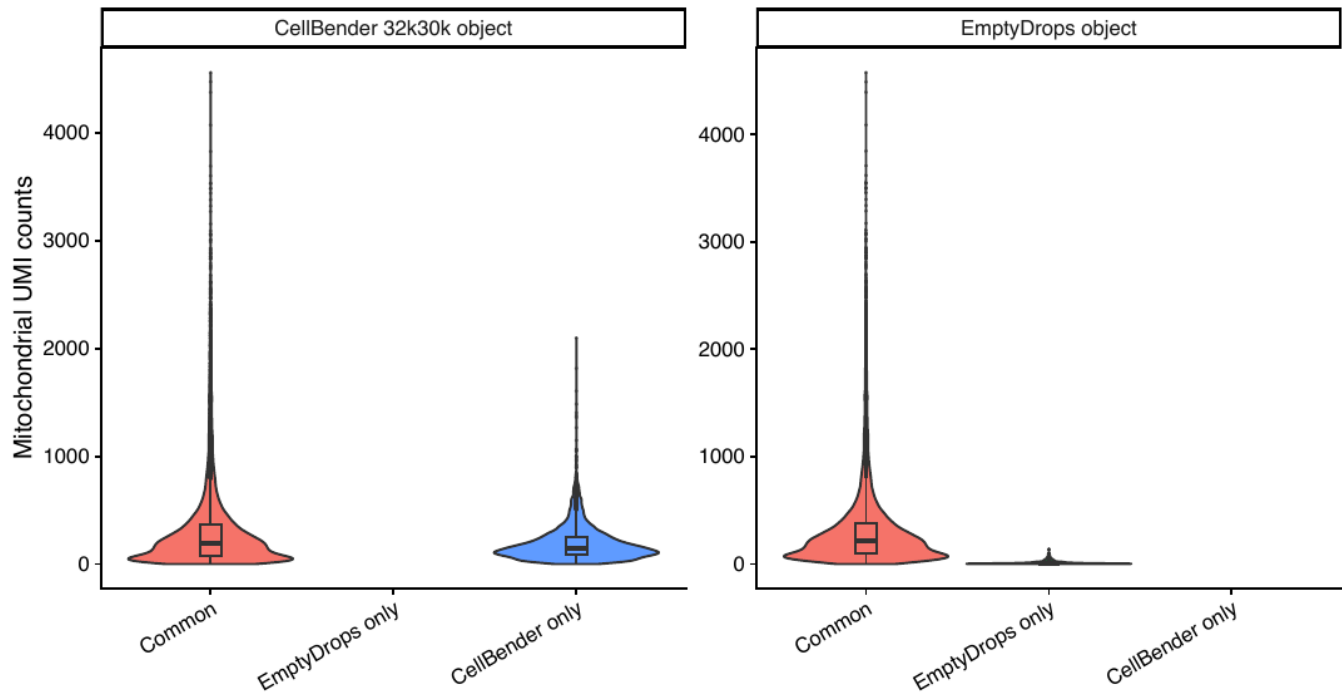
Percent mitochondrial per category - this is what identifies the dead cells behind the run 2 problem.



violin_mt_counts_by_barcode_category.pdf

Mitochondrial counts per category.

LK2-GEX: Mitochondrial UMI counts



5. In the pipeline

Both tools run for every sample in the Nextflow pipeline, and their overlap is recorded per run rather than being re-derived by hand.

emptyDrops / CellBender overlap per pipeline run

sample	n_emptydrops	n_cellbender	n_common	n_emptydrops_only	n_cellbender_only	pct_jaccard	run
LK1-GEX	23779	24115	22938	841	1177	91.9	260522
LK2-GEX	31201	38747	29852	1349	8895	74.5	260528
LK3-GEX	28968	32050	27404	1564	4646	81.5	260717

6. What was run, in order

Step	Script	What it does
01	01.emptydrops.R	emptyDrops on the raw matrix, FDR <= 0.01.
02	02.cellbender_run.sh	CellBender 0.3.0 in a container, expected + 15,000 droplets.
03	03.cellbender_run_30k_droplets.sh	The +30,000 droplet arm of the sweep.

Step	Script	What it does
04	<code>04.cellbender_qc.R</code>	Post-CellBender QC: knee plot, UMI before/after, percent background removed.
05	<code>05.compare_emptydrops_cellbender.R</code>	Barcode overlap, Venns at 25k and 32k, category breakdown.
06	<code>06.compare_emptydrops_cellbender_extended.R</code>	Adds the +30k arm, four-way Venn, and per-category QC.
07	<code>07.cell_calling_report.Rmd</code>	This report.

Two earlier attempts at the CellBender run are not kept: `scripts/2a.cellbender.sh` (2026-06-19, before containerisation) and `scripts/2b.cellbender_docker.sh` (2026-06-23, containerised but before the fix). Step 02 supersedes both.

Steps 05 and 06 are **both** kept, which is the exception to merging revisions into one step. They ran on different days and neither reproduces the other's output: step 05 wrote the 25k and 32k Venns on 2026-06-24, step 06 wrote the 32k30k arm, the four-way Venn and the QC violins on 2026-06-29.

7. Data locations

What	Location
emptyDrops	<code>results/emptydrops/<run>/<sample>/</code>
CellBender, +15k	<code>results/cellbender/<run>/<sample>/</code>
CellBender, +30k	<code>results/cellbender/<run>/<sample>_e30000/</code>
Comparison	<code>results/cellbender_comparison/<run>/<sample>/</code>
Pipeline	<code>results_nf/<run>/02_emptydrops/ ,</code> <code>.../02_cellbender/ , and</code> <code><run>_emptydrops_cellbender_overlap.csv</code>

8. Tools

Tool	Version	Source
DropletUtils emptyDrops	see session below	FDR ≤ 0.01
CellBender	0.3.0	<code>containers/cellbender_0.3.0.sif</code> , and <code>cellbender_0.3.0_fixed.sif</code>

```
## R version 4.5.1 (2025-06-13)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.5.2
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK version 3.12.1
##
## locale:
## [1] en_AU/en_AU/en_AU/C/en_AU/en_AU
##
## time zone: Australia/Melbourne
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] scales_1.4.0  ggplot2_4.0.2 knitr_1.51   purrr_1.2.1  stringr_1.6.0
## [6] readr_2.1.6   tidyr_1.3.2   dplyr_1.2.0
##
## loaded via a namespace (and not attached):
## [1] bit_4.6.0           gtable_0.3.6         jsonlite_2.0.0       crayon_1.5.3
## [5] qpdf_1.4.1          compiler_4.5.1        pdftools_3.9.0       Rcpp_1.1.1
## [9] tidysselect_1.2.1   parallel_4.5.1       jquerylib_0.1.4      yaml_2.3.12
## [13] fastmap_1.2.0       R6_2.6.1             generics_0.1.4       tibble_3.3.1
## [17] RColorBrewer_1.1-3  bslib_0.10.0         pillar_1.11.1        tzdb_0.5.0
## [21] rlang_1.1.7         cachem_1.1.0         stringi_1.8.7        xfun_0.56
## [25] S7_0.2.1            sass_0.4.10          bit64_4.6.0-1        otel_0.2.0
## [29] cli_3.6.5           withr_3.0.2          magrittr_2.0.4       digest_0.6.39
## [33] grid_4.5.1          vroom_1.7.0          askpass_1.2.1        hms_1.1.4
## [37] lifecycle_1.0.5     vctrs_0.7.1          evaluate_1.0.5       glue_1.8.0
## [41] farver_2.1.2        rmarkdown_2.30       tools_4.5.1          pkgconfig_2.0.3
## [45] htmltools_0.5.9
```

9. Caveats

1. **The full comparison exists for LK2 only.** CellBender was also run on LK1-resequenced, but `cellbender_comparison/` has just the one run, so the Venns and category QC are LK2's.
2. **The cell counts in the conclusions come from the meeting slides**, not from a saved summary table - 24,334 for run 1 and 31,198 against Cell Ranger's 32,846 for run 2.
3. **MT cutoffs are literature values**, listed for reference rather than derived here.